Two High-Impact AI Project Ideas

Designed to stand out from generic demos by solving real operational problems with data and AI engineering.

Project 1: Multi-Document Audit & Grounded RAG with Automated Hallucination Guardrails

Most AI portfolios include a naive Retrieval-Augmented Generation (RAG) chatbot that simply queries a vector database and blindly trusts whatever text comes out. A production-grade system evaluates its own answers, handles citations down to the paragraph level, and guards against hallucinations.

The Problem

Organizations (legal, finance, compliance) struggle to verify long reports (e.g., quarterly 10-Ks, policy documents, vendor contracts) quickly without risking false statements from AI.

The Architecture

Ingestion & Chunking: Hybrid chunking (semantic + markdown/table parsing) using tools like Unstructured or LlamaIndex.

Retrieval: Hybrid search combining dense embeddings (e.g., ChromaDB/pgvector) with sparse BM25 keyword matching and a re-ranking model (e.g., Cohere Rerank or BGE).

Self-Correction & Evaluation: Implement an evaluation layer (using frameworks like Ragas or TruLens) that scores context precision, context recall, and faithfulness before returning an answer to the UI.

What Proves You Are More Than a Vibe Coder

You can discuss chunking trade-offs, vector similarity metrics (Cosine vs. Dot Product), and re-ranking overhead.

You measure and log latency, token spend, and hallucination rates rather than just calling an API blindly.

Project 2: Real-Time Drift Detection & Predictive ML Engine with Feedback Loops

Pure prompt-based builders rarely understand classical machine learning lifecycles, streaming data, or what happens when real-world distributions shift over time. This project proves you understand end-to-end data systems.

The Problem

Predicting continuous real-world outcomes (e.g., real estate rental price shifts, delivery ETA delays, or e-commerce customer churn) where customer behavior and economic conditions constantly drift.

The Architecture

Data Pipeline: An automated pipeline using Python, Pandas, and DuckDB or SQLite that pulls live public data (via an API or scheduled batch) and handles outliers/missing values cleanly.

Model Core: Train a baseline tabular model (e.g., XGBoost or LightGBM) tracked with MLflow.

Drift & Performance Monitoring: Use an open-source monitoring library (like Evidently AI) to detect data drift and concept drift.

AI Copilot Layer: Connect an LLM agent that reads the drift reports and automatically synthesizes human-readable operational summaries: "Feature days_on_market drifted by 24% this week; model error increased by 8% in ZIP code 94103."

What Proves You Are More Than a Vibe Coder

You understand feature engineering, bias-variance tradeoffs, and train-test leakage.

You demonstrate how classical machine learning and modern LLMs integrate into a production feedback loop.